

ActKV: Efficient LLM Agents through Action-Guided KV Cache Management

Zihan Wang
University of Science and Technology
of China
Hefei, China
wangzh196@mail.ustc.edu.cn

Cheng Tang
University of Science and Technology
of China
Hefei, China
sisyphustc@mail.ustc.edu.cn

Lei Gong
University of Science and Technology
of China
Hefei, China
leigong0203@ustc.edu.cn

Chao Wang
University of Science and Technology
of China
Hefei, China
cswang@ustc.edu.cn

Wenqi Lou
Suzhou Institute for Advanced
Research, University of Science and
Technology of China
Suzhou, China
louwenqi@ustc.edu.cn

Teng Wang
Suzhou Institute for Advanced
Research, University of Science and
Technology of China
Suzhou, China
wangt635@ustc.edu.cn

Xuehai Zhou
University of Science and Technology
of China
Hefei, China
xhzhou@ustc.edu.cn

Abstract

Agentic LLM inference accumulates long KV caches across iterative observation-reasoning-action loops, imposing substantial memory overhead and limiting serving throughput. Existing compression methods emphasize overall output quality, overlooking the asymmetric importance of actions in driving task progress. Our key idea is to establish a compression criterion that values KV entries by their contribution to action generation and prioritizes action quality. However, iterative execution, dynamic memory demands, and scattered action-critical entries pose challenges to eviction policies, budget allocation, and paged memory integration. To this end, we propose ActKV, the first KV cache compression framework tailored for agentic LLM inference. (i) Action-oriented KV cache eviction exploits stable action access patterns to retain entries critical to future actions, supporting reliable task progress under compression. (ii) Confidence-driven adaptive budget allocation uses LLM’s intrinsic confidence to adapt the budget to evolving action-critical memory demands. (iii) Page-aware compression management standardizes compression into three primitives with customized kernels, realizing practical throughput gains. On long-trace tasks, ActKV retains an average of 98.53% of FullKV’s accuracy with only 25.98% of its peak KV cache memory. It also achieves 3.97× and 3.58× FullKV’s token and task throughput, delivering state-of-the-art performance.

Keywords: Agentic LLM Inference, KV Cache Compression

1 Introduction

Agents transform LLMs from passive question-answering into task-solving systems that perceive environments, reason, and act, reshaping how users interact with LLMs. The key is to introduce an intermediary agentic layer, forming a three-layer interaction architecture: user/environment, agentic framework, and LLM inference. The fundamental execution paradigm is iterative observation-reasoning-action loops, known as ReAct [36], which underpin modern agents like OpenClaw [21], Claude Code [1], and Codex [4]. As shown in Fig. 1, in each iteration, the agentic framework first collects environmental observations and extends LLM prompt. The LLM then reasons via chain-of-thought [29] for state understanding and deduction from accumulated traces. Finally, the LLM generates an action, which is executed by the agentic framework in environment. We view agents as complete systems formed by agentic frameworks and LLMs. Agentic LLM inference thus refers to the framework-driven inference process that repeatedly consumes observation tokens and generates reasoning and action tokens. *Importantly, observation and reasoning remain largely internal within the agent, and are often hidden or useless to users, while what users care about is the task progress driven by actions.*

Despite its potential for solving complex tasks, agentic inference incurs substantial KV cache overhead. Observation, reasoning, and action tokens accumulate as KV entries across interaction rounds, imposing heavy memory and computation costs on LLM serving systems. For example, a web shopping task with Qwen3-30B requires 25 iterations and about 35K tokens, consuming 3.20 GB of KV cache, with

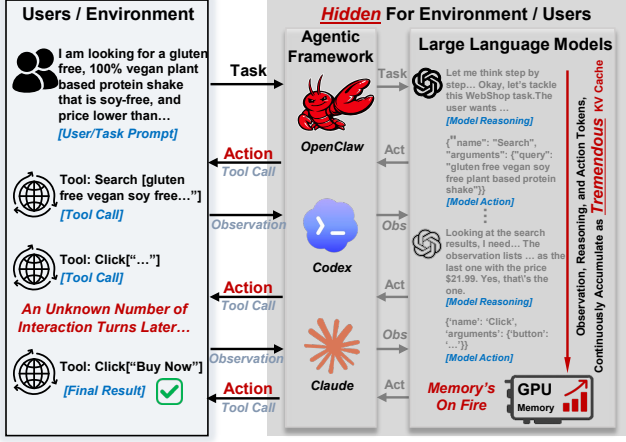


Figure 1. Agentic Workflow

observation and reasoning entries accounting for over 99%. This overhead grows with concurrent requests in batched serving, limiting concurrency and throughput. Moreover, long KV sequences can dilute attention and degrade model accuracy. Thus, KV cache compression is essential for efficient and accurate agent deployment.

KV cache compression dynamically evicts redundant KV entries during inference while preserving accuracy. Although many methods exist, most target conventional text generation and generalize poorly to agentic inference. **The fundamental mismatch lies in the generation objective: preserving overall output quality versus prioritizing action quality.** In continuous text generation, all tokens contribute to output completeness, so existing methods fairly maintain quality of each token generation. By approximating near-future attention, they greedily evict KV entries that are lowly attended by next few tokens [17, 31, 37]. **In agentic inference, however, token importance is asymmetric.** Observation and reasoning tokens mainly support internal state understanding and deduction, but are often hidden or useless for users. **Only action tokens affect environments and determine user task progress.** Thus, uniform token treatment is inherently misaligned with prioritizing action quality. Even worse, short-sighted retention for near-future reasoning can harm subsequent actions (Section 3.1).

Our key idea is to establish a **compression criterion that values each KV entry by its contribution to action generation and prioritizes action quality during compression**, thereby aligning eviction with the outcome-driven nature of agentic inference. However, three challenges arise.

What to keep? Iterative execution calls for stable criteria to identify action-critical KV entries. Since successive actions build on accumulated observations and reasoning, compression must preserve KV entries needed for both current and future actions. However, evolving context makes current importance an uncertain indicator of future utility. This raises

a fundamental question: do action-critical KV entries exhibit consistent attention patterns across iterations? Such regularities could provide a reliable basis for compression while preserving subsequent action quality.

How much to keep? Dynamic memory demands of action-critical KV entries require adaptive budget allocation. The KV cache budget controls the trade-off between memory footprint and generation quality. Across task types and difficulty levels, the number of action-critical KV entries varies with interaction rounds, observation lengths, and reasoning complexity [36]. However, existing methods typically profile full-KV memory consumption on specific tasks and apply predefined compression ratios during inference [3, 10]. Such budgets derived from workload-specific offline profiling fail to adapt to evolving action-critical memory demands.

How to compress efficiently? Scattered action-critical KV entries require reconciling compression with paged memory management. Agent deployment relies on serving systems (e.g., vLLM [27] and SGLang [39]), which use block-based KV management to reduce fragmentation and improve throughput. However, scattered action-critical entries require token-level eviction misaligned with block-level allocation. Most implementations remain limited to algorithmic validation. The few system-level approaches incur copying overhead offsetting compression gains [2] or rely on policy-specific adaptations lacking generality [23]. These limitations hinder practical memory savings from fine-grained eviction.

To address the challenges, we propose ActKV, the *first* KV cache compression framework tailored for agentic LLM inference, enabling accurate, memory-efficient, and high-throughput agent deployment through three components.

Action-oriented KV cache eviction prioritizes action generation quality during compression. We observe that actions generated within a given task exhibit similarities. First, each task involves a finite set of semantically related action types. Second, action arguments repeatedly draw on shared task context. Third, action outputs follow a standardized template [9]. Motivated by these semantic, contextual, and structural similarities, we examine attention maps and observe that action-critical KV entries form a stable subset across iterations but contribute intermittently (Section 3.1). Accordingly, we propose attention-aware Least Recently Frequently Used (LRFU) to retain entries critical to future actions, enabling reliable task progress with a low memory footprint.

Confidence-driven adaptive budget allocation dynamically adjusts the KV cache budget during inference. We observe a strong correlation between the LLM’s intrinsic confidence and budget sufficiency. Tighter budgets make action-critical KV information incomplete, forcing model to extend reasoning to recover missing context and causing confidence drops (Section 3.2). Accordingly, we design a confidence monitor that combines sliding-window analysis with linear fitting

to detect confidence degradation caused by budget insufficiency and trigger budget adjustments, adapting to evolving action-critical memory demands.

Page-aware compression management bridges fine-grained KV eviction with block-based memory. We observe that the eviction-based compression can be distilled into a unified workflow of three primitives: attention calculation, entry eviction, and cache compaction, with calculation and compaction as the main system gaps (Section 3.3). Accordingly, we design customized kernels. The recovery-based attention calculation kernel reconstructs attention scores from softmax log-sum-exp (LSE), while the in-place cache compaction kernel performs one-shot in-place copying by conflict-free slot planning. These standardized primitives and customized kernels translate algorithmic gains into tangible memory savings and throughput improvements.

We evaluate ActKV on four agentic LLMs, Qwen3-30B, Qwen3-235B [33], GPT-OSS-20B, and GPT-OSS-120B [20], across HotpotQA [34], WebShop [35], and ALFWorld [25]. Relative to FullKV, ActKV averages 98.53% accuracy with only 25.98% peak KV cache memory on long-trace tasks, compared with at most 74.22% accuracy and about 33.60% memory for StreamingLLM [31], SnapKV [17], and R-KV [2]. End to end, ActKV improves token and task throughput over FullKV by 3.97 $\times$ and 3.58 $\times$, respectively.

- **Action-oriented KV cache eviction.** We prioritize action quality during compression to support reliable task progress under aggressive memory constraints.
- **Confidence-driven adaptive budget allocation.** We use LLM’s intrinsic confidence to adapt the budget to evolving action-critical memory demands.
- **Page-aware compression management.** We standardize compression into three primitives with customized kernels, realizing practical throughput gains.
- While maintaining or even improving accuracy, ActKV reduces memory footprint and improves throughput, achieving *state-of-the-art* performance.

2 Background and Motivation

2.1 Agentic LLM Inference

Agents have reshaped human interaction with large language models (LLMs). Unlike conventional question-answering, agents introduce a framework between users/environments and the LLM to repeatedly invoke the model, parse its outputs, and execute actions on behalf of the user. The framework follows iterative observation-reasoning-action loops, known as ReAct [36], enabling complex task solving in systems like OpenClaw [21], Claude Code [1], and Codex [4]. Observations provide environmental feedback, reasoning captures internal deduction, and actions provide an explicit interface to affect environments: document search in retrieval-based question answering, button clicks in web tasks, object manipulation in embodied tasks.

Agentic LLM inference thus refers to framework-driven inference that repeatedly consumes observations and generates reasoning and actions. It differs from conventional inference in two key aspects. First, interaction traces accumulating across iterations rapidly expand the KV cache, increasing memory costs while diluting attention and degrading accuracy. Second, token importance is highly asymmetric: observation and reasoning tokens primarily support internal understanding and deduction but are often hidden from or less useful to users, whereas action tokens directly affect environments and determine task progress.

These distinctions respectively highlight the need for KV cache compression and action-oriented optimization in agentic LLM inference.

2.2 Context Engineering

Context engineering is an important optimization in agentic frameworks. It reduces the number of tokens entering the model at the text level, thereby mitigating context-window pressure and indirectly slowing KV cache growth. Most context engineering operations preserve the existing trace prefix, so KV cache can still be continuously appended. For example, memory-based methods store important information and retrieve it as additional observations when needed [6, 13, 32]. Some operations, such as LLM-based summarization [14, 18], rewrite historical context and affect KV cache reuse, but they incur re-prefilling overhead and are typically triggered only occasionally [19], e.g., when the context window approaches its limit. In contrast, eviction-based KV cache compression operates at runtime inference. It does not modify high-level textual context, but reduces storage and computation costs by removing redundant KV entries.

In summary, context engineering and KV cache compression are orthogonal optimizations. Context engineering decides what text enters the model, while KV cache compression decides which runtime KV states are worth preserving.

2.3 KV Cache Compression

During autoregressive LLM generation, each new token attends to all previous tokens, whose intermediate states are stored as Key-Value (KV) cache to avoid recomputation [5]. As sequences grow, the KV cache expands proportionally, causing substantial memory and computation overhead and making KV cache compression essential [38]. Eviction-based compression removes redundant KV entries, with attention scores serving as the gold-standard importance signal [2, 17, 37]. Formally, for layer ℓ and attention head h , at decoding step i , the current hidden state is projected into query $q_i^{\ell,h}$, while each previous token s has cached key-value pair $(k_s^{\ell,h}, v_s^{\ell,h})$. Let $K_{<i}^{\ell,h} = [k_1^{\ell,h}, \dots, k_{i-1}^{\ell,h}]$ and $V_{<i}^{\ell,h} = [v_1^{\ell,h}, \dots, v_{i-1}^{\ell,h}]$. The attention score and output are:

$$A_i^{\ell,h} = \text{softmax} \left(q_i^{\ell,h} (K_{<i}^{\ell,h})^\top \right), \quad o_i^{\ell,h} = A_i^{\ell,h} V_{<i}^{\ell,h},$$

where $A_i^{\ell,h} \in \mathbb{R}^{1 \times (i-1)}$. A larger $A_i^{\ell,h}[s]$ indicates that the cached KV entry of token s contributes more to current token generation, and is therefore more critical for preserving the quality of the current output.

Existing KV cache compression methods generally assume uniform token importance and use near-future attention to greedily evict entries less useful for upcoming tokens. StreamingLLM [31] preserves initial and recent tokens, H2O [37] and SnapKV [17] approximate near-future attention through score accumulation or observation windows. During reasoning, R-KV [2] deduplicates similar entries, whereas CrystalKV [28] prioritizes those receiving long-range attention. However, neither readily generalizes to multi-round agentic inference.

Agentic workflows are outcome-driven, where final actions matter more than intermediate reasoning. KV entries useful for near-future reasoning may not support later action generation, making existing methods shortsighted and degrading action correctness and stability under compression.

2.4 Budget Allocation

The KV budget controls compressed-cache capacity and defines the trade-off between memory footprint and generation quality. Existing methods typically use workload-specific, offline-profiled static budgets. This is impractical for agentic inference, where the number of action-critical KV entries varies with task type, task difficulty, and model stochasticity. Different environments produce observations of different lengths, difficult tasks require more interaction rounds and longer reasoning, and stochastic actions may lead to divergent future contexts. Thus, a fixed budget may waste memory or over-compress the cache, degrading generation quality.

Some methods study budget allocation, but mainly distribute a predefined total budget across layers or heads. For example, PyramidKV [3] assigns larger budgets to shallow layers with dispersed attention and smaller budgets to deeper layers with focused attention, while Ada-KV [10] extends

this idea to head-wise allocation. These methods optimize relative allocation within a fixed budget, but do not determine the overall budget for dynamic agentic workflows.

2.5 Paged Memory Management

Block-based paged KV cache management is common in modern LLM serving systems, with PagedAttention [16] as representative design adopted by vLLM [27] and SGLang [39]. It organizes KV cache memory into fixed-size physical blocks:

$$\text{PagedKVCache}[2, B, BS, H, D]$$

where the first dimension represents key and value states, B is the number of physical blocks, BS is the number of slots per block, H is the number of KV heads, and D is the head dimension. A block table maps logical token positions to physical blocks. For example, when $BS = 2$, $\text{block_table} = [17, 9]$ and $\text{sequestered_k} = 3$ correspond to occupied physical slots $\text{slot_map} = [34, 35, 18]$, leaving one unused slot in last block. By reusing fixed-size blocks, paged KV management avoids per-request contiguous allocation, reducing fragmentation and improving utilization and throughput.

However, action-critical entries are scattered across blocks, making token-level eviction misaligned with block-level allocation. Existing solutions remain limited. R-KV [2] gathers retained entries into an additional contiguous buffer, increasing peak memory usage and offsetting compression benefits. ThinKV [23] modifies block-table metadata for its compression policy, but algorithm-specific changes disrupt standard serving workflows and limit generality and scalability.

Thus, fine-grained KV compression requires general and scalable integration with paged memory management to translate token-level eviction into actual memory savings and throughput gains.

3 Key Insight

3.1 Action-Preferred Entries Are Distinct and Stable

We analyze the attention patterns of KV entries that contribute to action generation. As shown in Fig. 2, the attention

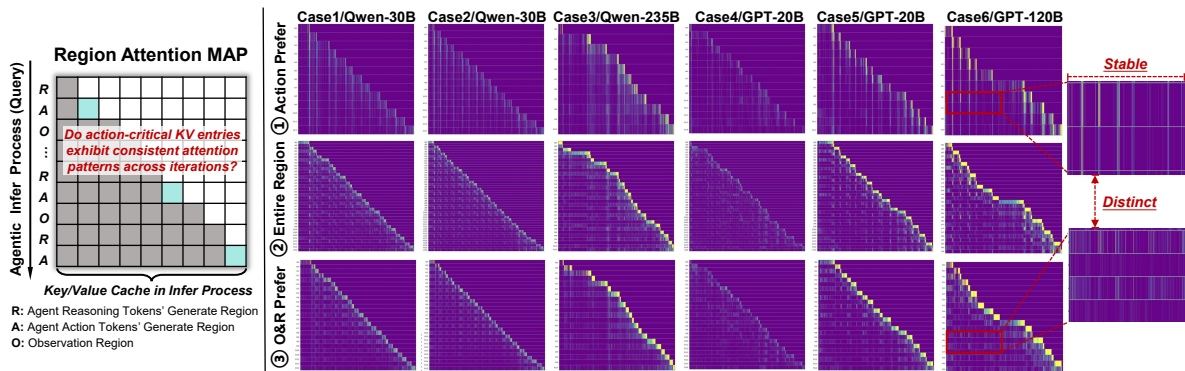


Figure 2. Action-Oriented KV Attention Pattern

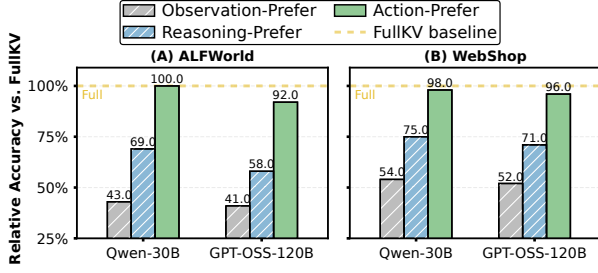


Figure 3. Action-Preferred KV Preservation Study

map places key/value states on the horizontal axis and query states on the vertical axis. Query states are partitioned into interleaved observation (O), reasoning (R), and action (A) regions. For each region, we average attention scores along the query dimension to measure each KV entry’s contribution, where brighter colors indicate higher scores and stronger preference. We then extract the top 30% preferred KV entries for action regions, and apply same procedure to observation and reasoning regions. Across Qwen3-30B, Qwen3-235B, GPT-OSS-20B, and GPT-OSS-120B on ALFWorld and WebShop, we consistently observe two patterns.

First, **action-critical KV entries are distinct from those serving observation and reasoning tokens**. Observation tokens mainly attend to local extended-prefill context. Reasoning tokens aggregate information across all previous observations, producing diffuse attention over history. Action tokens focus on a smaller set of task-critical history for executable decisions. Thus, KV entries important for actions may rank low for observation or reasoning, causing observation or reasoning guided eviction to discard action-critical entries. This explains why normal long-context compression methods fail in agentic inference.

Second, **action-critical KV entries are stable across iterations**. We observe that actions generated within a given task exhibit similarities in their semantics, contextual dependencies, and output structure. Each action consists of three components: type, arguments, and format [24]. First, each task involves a limited set of semantically related action types, such as clicking in web interaction, object pickup and placement in embodied control, and search and lookup in information retrieval. Second, action arguments repeatedly draw on shared task context, including task constraints, relevant entities, and evidence acquired in previous iterations. Third, action outputs in modern agentic LLMs are often constrained to a common template [9]. Motivated by these semantic, contextual, and structural similarities, we examine attention maps during action generation and find that action-critical KV entries form a stable subset across iterations (quantified in Section 5.1). This stability makes current action attention an effective signal for preserving KV entries useful for subsequent action generation.

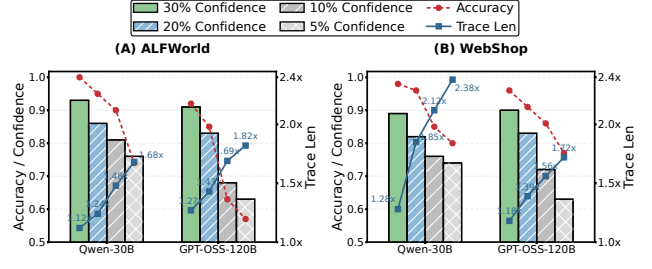


Figure 4. Confidence-Budget Correlation Study

We validate this with a controlled compression experiment. From 100 full-KV-correct traces, we remove the last three iterations’ KV cache and compress the remaining prefix KV to 30% using observation-, reasoning-, or action-preferred scores. The agent continues for at most six iterations. As shown in Fig. 3, action-preferred preservation achieves the best success rate, confirming that action-preferred KV entries are visually distinct, temporally stable, and directly predictive of the entries needed for correct downstream actions.

3.2 Confidence Reveals KV Budget Sufficiency

We analyze the correlation between LLM confidence and KV cache budget. Confidence is derived from the model’s predicted token distribution, where peaked distributions indicate certainty and flatter distributions indicate uncertainty (detailed in Section 6.1). We take 100 full-KV-correct traces, remove KV cache of last three iterations, and compress remaining prefix KV cache to 30%, 20%, 10%, and 5% by action preference. We then compare success rate, confidence ratio, and trace-length ratio against full KV. As shown in Fig. 4, tighter budgets consistently reduce success rate and confidence while increasing trace length.

We further inspect the generated text traces to understand this behavior. Under tight budgets, compression corrupts action-critical information in KV cache. The model then exhibits low-confidence linguistic markers, such as "wait", "I guess", and "let me check", suggesting that it attempts to extend its reasoning to restate missing context and recover task progress. However, longer reasoning further stresses limited budgets, causing repeated restatement of incomplete history and eventually re-exploration or failure.

This reveals that confidence reflects budget sufficiency in agentic inference. **As the budget tightens, action-critical KV information becomes incomplete. The model compensates by extending reasoning to recover missing context, which manifests as confidence degradation.** Therefore, confidence degradation can serve as a reliable online signal for guiding adaptive budget adjustment during inference.

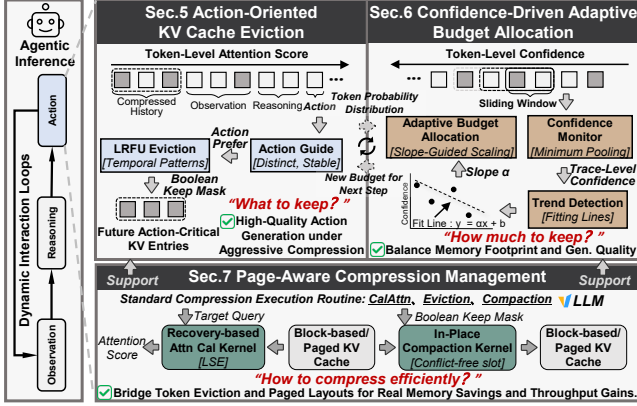


Figure 5. Overview of ActKV

3.3 Standardized Page-Aware KV Compression

We distill a unified workflow that integrates token-level KV compression with paged memory management. We observe that most compression methods, including H2O [37], StreamingLLM [31], SnapKV [17], R-KV [2], RaaS [12] and ActKV, can be standardized into three primitives:

$$\begin{aligned} \text{AttnScore} &\leftarrow \text{CalAttn}(\text{targetq}, \text{KCache}), \\ \text{KeepMask} &\leftarrow \text{Eviction}(\text{AttnScore}, \text{Budget}), \\ \text{KVCache} &\leftarrow \text{Compaction}(\text{KVCache}, \text{KeepMask}). \end{aligned}$$

Attention calculation computes scores between target queries and cached keys, where different methods use different targets, e.g., observation-window queries in SnapKV and action-specific queries in ActKV. Entry eviction applies a policy over scores and budget to produce a Boolean keep mask, with 1 for retention and 0 for eviction. Cache compaction removes evicted entries and reorganizes remaining KV cache for the subsequent inference.

This decomposition shows that eviction is cache-layout agnostic, while the main system challenges lie in attention calculation and cache compaction. For attention calculation, paged attention backends do not expose full attention scores, whereas standard attention kernels require scattered pages to be materialized into contiguous memory. We address this with a recovery-based attention calculation kernel that reconstructs scores from softmax log-sum-exp (detailed in Section 7.1). For cache compaction, arbitrary token eviction creates holes across pages. Gather-based compaction incurs extra memory, while naive in-place compaction suffers from read-write conflicts and cannot be safely parallelized. We address this with conflict-free slot planning and a one-shot in-place compaction kernel (detailed in Section 7.2).

Thus, **standardizing KV compression into a unified workflow supported by customized kernels provides a general approach to integrating token-level eviction with paged memory management**, turning fine-grained compression into practical memory savings and throughput gains.

4 Overview

Fig. 5 presents ActKV, a KV cache compression framework tailored for agentic LLM inference. ActKV consists of three key modules: action-oriented KV cache eviction, confidence-driven adaptive budget allocation, and page-aware compression management. At the end of each observation, reasoning, action round, the action-oriented eviction compresses KV cache to target budget. Guided by action-region attention scores and attention-aware LRFU policy, it evicts entries that are less important for subsequent action generation, enabling reliable task progress with a low memory footprint (Section 5). The confidence-driven adaptive budget allocation continuously monitors model confidence from predicted token distributions via sliding-window scheme. Before each compression step, it detects confidence trend via linear fitting and adjusts budget accordingly, adapting to evolving action-critical memory demands across tasks (Section 6). The page-aware compression management bridges compression policies with block-based KV memory layouts via recovery-based attention calculation and in-place compaction kernels, translating fine-grained compression into practical memory savings and throughput gains (Section 7).

Overall, eviction and budget allocation module form a lightweight feedback loop between generation behavior and KV cache management, while page-aware compression module provides system-level interfaces for efficiently executing compression policies on paged memory layouts. These modules work synergistically to enable accurate, memory-efficient, and high-throughput agent deployment.

5 Action-Oriented KV Cache Eviction

5.1 Action Access Pattern and Temporal KV Eviction

In Section 3.1, we qualitatively observed that action-preferred KV entries remain stable across successive action regions. We now quantify this stability with the action access vector (AAV) for each KV entry:

$$\text{AAV}_{kv} = [a_1, a_2, a_3, \dots, a_n], \quad kv \in \text{KV_Cache},$$

where n is the number of action regions that can attend to this entry. Each a_i indicates whether the entry contributes to i -th action region: $a_i = 1$ if its attention score falls within top- p 90% of that region, and $a_i = 0$ otherwise. We discard all-zero AAVs, as they never contribute to action generation.

We classify the remaining AAVs into three mutually exclusive categories. A *cold-start* AAV begins with one or more zeros, indicating that the entry becomes important only in later actions. An *intermittent* AAV contains zeros between its first and last one, indicating discontinuous contribution. A *continuous* AAV contains no zero between its first and last one, indicating consecutive contribution. Fig. 6 reports both the entry-count ratio and per-access attention-score ratio of these categories. Cold-start entries are negligible, suggesting

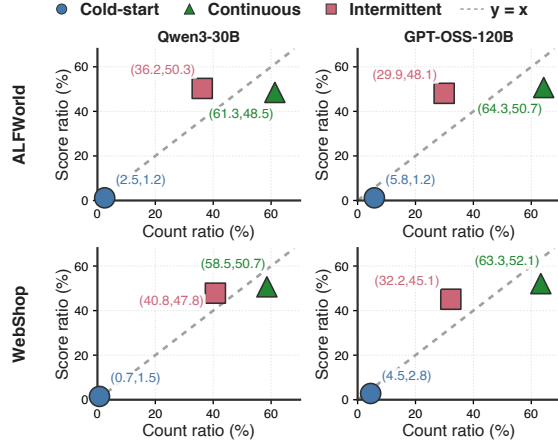


Figure 6. Access Patterns of Action-Preferred KV Entries

that entries preferred by current actions are usually predictable from previous actions. Continuous entries are the most common, but their per-access attention contribution is lower than that of intermittent entries, which contribute strong attention in a discontinuous manner. Therefore, evicting entries solely based on current action preference may mistakenly remove entries that will become important again.

Motivated by the patterns, we shift from static importance to temporal KV utility. The key criterion is: *a KV entry should be evicted only after it has not contributed high attention for a sufficiently long period*. We instantiate this with attention-aware Least Recently Frequently Used (LRFU) policy. For each KV entry e_i , its keep score (KS) is updated as:

$$KS_t(i) = \lambda \cdot KS_{t-1}(i) + \text{AttnScore}_t(i)_{\{\text{hit at } t\}},$$

where t is the compression step, $\lambda \in [0, 1]$ is the decay factor, and $\text{AttnScore}_t(i)$ is the action-region attention score of e_i . An entry is considered hit if its action-region attention score falls within the top- p set, with $p = 90\%$.

This formulation balances recency and frequency. At $\lambda = 0$, recency-based eviction may discard temporarily inactive but future-important intermittent entries. At $\lambda = 1$, frequency-dominated eviction may retain obsolete continuous entries. With $0 < \lambda < 1$, high-attention intermittent entries survive temporary inactivity, while score decay enables eviction of stale continuous entries.

5.2 Eviction Algorithm

Algorithm 1 summarizes the action-oriented eviction procedure. We identify the action region by matching generated tokens to the LLM’s standardized output template to detect the start and end of action generation [15, 22]. We collect attention scores within this region and use them to update the temporal utility of KV entries after each ORA round. Action-critical KV entries differ from those supporting observation and reasoning and remain stable across iterations.

Algorithm 1 Action-Oriented KV Cache Eviction

Inputs: B: KV budget, p : top- p threshold for hit mask, λ : keep-score decay rate, Q^{act} : action-region queries, K/V: key/value cache, KS: keep scores of existing KV entries, L_{ora} : number of newly generated KV entries in the current ORA round.

Outputs: K'/V' : compressed KV cache, KS' : updated keep scores.

```

/* Triggered At The End Of */
/* Each Observation-Reason-Action (ORA) Round */
/* Step 1: Get Action-Region Preference */
1:  $\text{AttnScore} \leftarrow \text{PAGEDCALATTN}(Q^{\text{act}}, K)$ 
/* Step 2: Init Keep Scores for New ORA */
2:  $KS_{\text{new}} \leftarrow \text{INIT}(L_{\text{ora}}, 0)$ 
3:  $KS \leftarrow \text{CAT}(KS, KS_{\text{new}})$ 
/* Step 3: Identify Hit and Miss KV Entries */
4:  $M_{\text{hit}} \leftarrow \text{GETHITMASK}(\text{AttnScore}, p)$ 
5:  $M_{\text{miss}} \leftarrow 1 - M_{\text{hit}}$ 
/* Step 4: Update Keep Scores */
6:  $KS[M_{\text{hit}}] \leftarrow \lambda \cdot KS[M_{\text{hit}}] + \text{AttnScore}[M_{\text{hit}}]$ 
7:  $KS[M_{\text{miss}}] \leftarrow \lambda \cdot KS[M_{\text{miss}}]$ 
/* Step 5: Select Retained KV Entries */
8:  $\text{KeepMask}, KS' \leftarrow \text{TOPK}(KS, k = B)$ 
/* Step 6: KV Cache Compaction */
9:  $K'/V' \leftarrow \text{PAGEDCOMPACTION}(K/V, \text{KeepMask})$ 
10: return  $K', V', KS'$ 

```

Thus, current action attention provides an effective signal for identifying KV entries useful for future actions.

The algorithm updates each KV entry’s keep score using an attention-aware LRFU rule, favoring eviction of entries that have not received high attention for an extended period. This update protects intermittent entries that temporarily become inactive but later receive high per-access attention, while score decay allows stale continuous entries to be evicted. Finally, entries with the highest keep scores are retained, and the KV cache is compacted. This policy exploits stable action access patterns to preserve information critical to future actions under a constrained budget.

6 Confidence-Driven Budget Allocation

6.1 Confidence Monitor

Section 3.2 shows that insufficient KV budgets corrupt action-critical information, forcing model to extend reasoning to recover missing context and causing confidence degradation. To detect such budget-induced degradation during inference, we introduce a confidence monitor based on internal predicted token distributions [11]. The monitor operates at three levels: token-level confidence, trace-level confidence, and trend detection across generation steps.

Token-level confidence. Given the predicted token distribution P_i at position i , we define token-level confidence as the

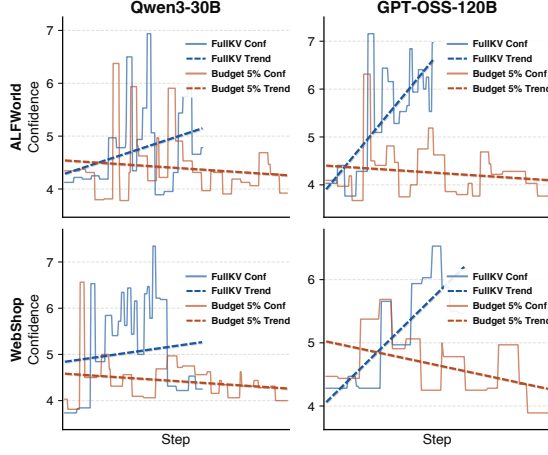


Figure 7. Effectiveness of Confidence Monitor

negative average log-probability of the top- k tokens:

$$T_i = -\frac{1}{k} \sum_{j=1}^k \log P_i(j),$$

where $P_i(j)$ denotes the probability of the j -th largest token and k is the number of top tokens considered. High confidence corresponds to peaked distributions and greater model certainty, while low confidence indicates uncertainty in token prediction.

Trace-level confidence. Token-level confidence is local and may fluctuate across tokens, so we aggregate it to characterize the overall confidence of a generated trace. Given a token-level confidence sequence $\{T_i\}_{i=1}^L$, we slide a window of size w with stride s and apply minimum pooling:

$$R'_m = \min_{i \in [ms+1, \dots, ms+w]} T_i, \quad m = 0, 1, \dots, \left\lfloor \frac{L-w}{s} \right\rfloor.$$

This produces an intermediate confidence sequence R' . We then retain the bottom $b\%$ values of R' as the final trace-level confidence sequence:

$$R = \{r \in R' | r \leq Q_b(R')\},$$

where $Q_b(R')$ denotes the b -th percentile of R' . Minimum pooling and bottom- $b\%$ selection jointly emphasize low-confidence segments, making the monitor sensitive to localized confidence degradation. Such degradation often occurs when the model restates missing context, repairs its reasoning, or recovers task progress with phrases such as “wait”, “I guess”, or “let me check”.

Trend detection. Trace-level confidence can still fluctuate, making absolute values unreliable. We therefore estimate its overall trend by applying first-order least-squares fitting to R . Let α denote the fitted slope. If $\alpha < 0$, confidence is decreasing, suggesting an insufficient KV budget, so ActKV increases the budget to preserve more entries. The monitor

Algorithm 2 Confidence-Driven Budget Allocation

Inputs: P : predicted distribution, k : number of top tokens, TL : token confidence list, ws : sliding-window size, s : stride, b : bottom- $b\%$ ratio, B : current KV budget, $scale_{up}$: budget scaling factor, B_{upper} : budget upper bound.

Outputs: B' : updated KV budget.

```

/* Monitoring in Parallel with LLM Inference */
/* Step 1: Collect Token-Level Confidence */
1:  $T \leftarrow GETTOKENCONF(P, k)$ 
2:  $TL \leftarrow APPEND(TL, T)$ 
/* Step 2: Estimate Trace-Level Confidence */
3:  $R \leftarrow GETTRACECONF(TL, ws, s, b)$ 
/* Step 3: Detect Confidence Trend */
4:  $\alpha \leftarrow FITLINE(R)$ 
/* Step 4: Allocate Budget Before Compression */
5:  $B' \leftarrow B$ 
6: if TriggerCompress then
7:   if  $\alpha < 0$  then  $B' \leftarrow \min(B \cdot scale_{up}, B_{upper})$ 
8: return  $B'$ 

```

runs in parallel with model inference, introducing negligible overhead to LLM inference.

As shown in Fig. 7, we apply above monitor to experiments in Section 3.2 and compare the confidence trend under a 5% KV cache budget with that under full-KV cache setting. The monitor accurately captures the confidence degradation caused by insufficient budgets, while the linear fitting effectively reflects the overall trend despite local fluctuations.

6.2 Adaptive Budget Allocation Algorithm

Algorithm 2 presents the confidence-driven adaptive budget allocation procedure. It continuously tracks generation confidence in parallel with LLM inference and adjusts the KV cache budget before compression. The algorithm first derives token-level confidence from predicted token distributions and appends it to a running sequence. It then aggregates token-level confidence into trace-level confidence using sliding-window minimum pooling and bottom-percentile selection, emphasizing localized degradation associated with context restatement, reasoning repair, and task-progress recovery. Finally, it fits a linear function to detect the confidence trend and increases the budget when confidence declines, forming a lightweight feedback loop between generation behavior and KV cache management.

As the task progresses, action-critical KV entries accumulate, increasing the memory required to support subsequent actions. We therefore initialize the cache with a small budget and expand it only when confidence declines, accommodating growing memory demands while avoiding premature over-allocation. When confidence stabilizes or recovers, we retain the current budget, as this improvement suggests that the allocation is sufficient but does not establish that a smaller budget would suffice. Maintaining the allocation thus

Algorithm 3 Recovery-Based Attention Calculation Kernel

Inputs: Q: current query states, PagedKV: paged KV cache, BlockTable: logical-to-physical block mapping, LSE: softmax log-sum-exp, GQAGroup: query-head group for each KV head, ActionReq: requests in action regions.

Outputs: Out: running mean attention scores within action region.

```

/* Triggered For PagedCalAttn Operation */
/* Step 1: Parallel Over ActionReq/Block/Head */
1: parallel for req ∈ ActionReq, block, kv_head do
/* Step 2: Load Paged KV Block */
2: id ← BlockTable[req, block]
3: K_block ← PagedKV[0, id, :, kv_head, :]
/* Step 3: Recover for Current Action Query */
4: for q_head ∈ GQAGroup(kv_head) do
5: Q_head ← LOADQUERY(Q, req, q_head)
6: Logits ← Q_head · K_blockT
7: Attn ← exp(Logits − LSE[req, q_head])
/* Step 4: Aggregate over Query Heads */
8: AttnScore ← max(AttnScore, Attn)
/* Step 5: Update Action-Region Attention */
9: UPDATEACTIONATTN(Out, AttnScore)
10: return Out

```

avoids renewed information loss from budget reductions and supports continued confidence recovery.

7 Page-Aware Compression Management

7.1 Recovery-Based Attention Calculation Kernel

In Section 3.3, we formulate token-level KV cache compression as a unified workflow consisting of three primitives: attention calculation, entry eviction, and cache compaction. Attention calculation is a key bottleneck in adapting fine-grained compression to paged memory management. Paged attention backends, such as paged FlashAttention [7], support non-contiguous KV blocks but do not expose the full attention matrix needed for eviction. Standard attention kernels provide these scores but require gathering scattered KV blocks into contiguous memory, incurring costly copies and temporary storage.

To avoid recomputing attention from scratch, *our key idea is to recover attention scores from the softmax log-sum-exp (LSE) produced by paged attention*. LSE is a by-product of on-the-fly softmax normalization. For layer ℓ and head h , at decoding step i , given query $q_i^{\ell,h}$ and cached keys $K_{<i}^{\ell,h} = k_{j,j<i}^{\ell,h}$, the LSE is:

$$\text{LSE}_i^{\ell,h} = \log \sum_{j<i} \exp(s_{i,j}^{\ell,h}), \quad s_{i,j}^{\ell,h} = q_i^{\ell,h} (k_j^{\ell,h})^\top.$$

where $s_{i,j}^{\ell,h}$ is the unnormalized attention logit. Since attention is obtained by softmax normalization, each attention score

Algorithm 4 In-Place KV Cache Compaction Kernel

Inputs: PagedKV: paged KV cache, KeepMask: keep mask, SlotMap: logical-to-physical slot mapping, SeqUsedK: used KV length of reqs, B: KV budget of reqs, BlockSize: page block size.

Outputs: PagedKV: compacted paged KV cache.

```

/* Triggered at Action Region Exit */
/* Step 1: Plan Copy Pairs in Parallel */
1: parallel for req ∈ ActionExitedRequests do
2: Y ← [SeqUsedK[req] / BlockSize]
3: X ← [B[req] / BlockSize]
4: Start ← (Y − X) · BlockSize
5: End ← Start + B[req]
/* Step 2: Identify Logical Destination Slots */
6: DstMask ← ¬KeepMask[req, Start : End]
/* Step 3: Identify Logical Source Slots */
7: KeepMask[req, Start : End] ← False
8: SrcMask ← KeepMask[req]
/* Step 4: Map Logical Slots to Physical Slots */
9: DstSlots ← SlotMap[req, DstMask]
10: SrcSlots ← SlotMap[req, SrcMask]
/* Step 5: Generate Conflict-Free Copy Pairs */
11: CopyPairs[req] ← ZIP(SrcSlots, DstSlots)
/* Step 6: Parallel One-Shot In-Place Copy */
12: parallel for (src, dst) ∈ CopyPairs do
13: PagedKV[:, dst] ← PagedKV[:, src]
14: return PagedKV

```

can be recovered as:

$$A_{i,j}^{\ell,h} = \exp(s_{i,j}^{\ell,h} - \text{LSE}_i^{\ell,h}).$$

Thus, during compression, we only recompute local query-key dot products within each paged KV block and normalize them with LSE already produced by paged attention backend.

Algorithm 3 presents our recovery-based attention calculation kernel under grouped-query attention (GQA). We identify action boundaries by matching generated tokens against the LLM’s standardized output template [15, 22]. The kernel processes only requests currently generating actions and parallelizes over these requests, logical KV blocks, and KV heads. Each thread block handles one KV head of one logical block for one request, resolves its physical block through the block table, and directly loads keys from the paged KV cache. For each associated query head, it computes logits for the current query token and recovers attention probabilities using the corresponding LSE. It then takes the maximum across query heads in the GQA group and incrementally updates the mean across action query tokens, producing a score for each KV entry without buffering past queries. The accumulated state is reset to zero upon entry into each action region. Implemented in Triton [26], this kernel computes token-level attention scores directly on paged KV layouts without gathering scattered blocks into contiguous memory.

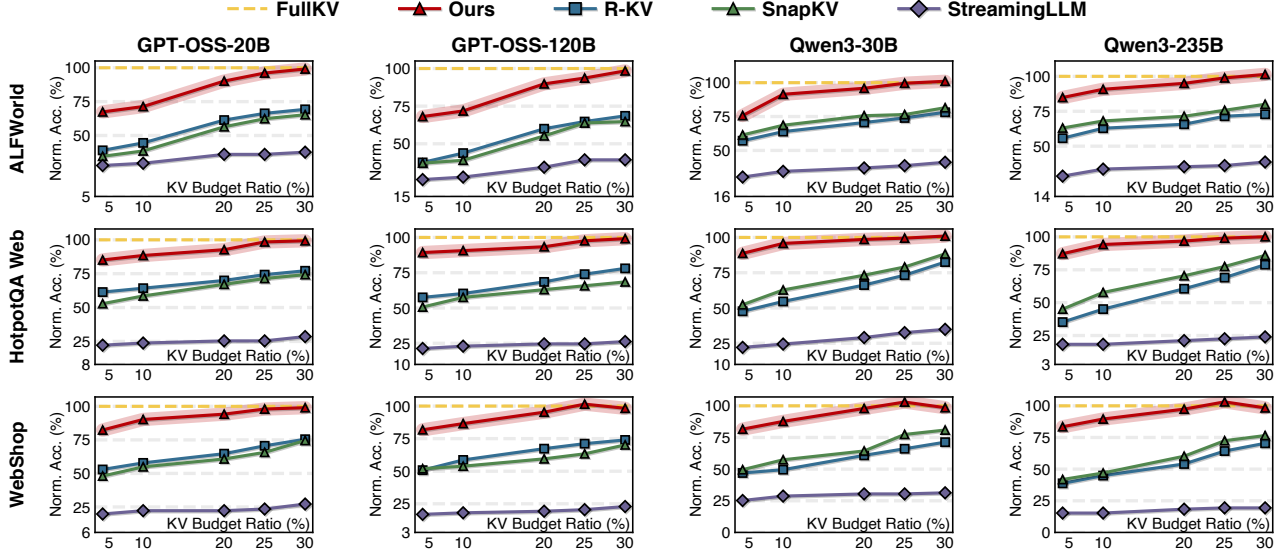


Figure 8. Accuracy Comparison

7.2 In-Place Compaction Kernel

Cache compaction is another key bottleneck in bridging token-level KV compression with paged memory management. Token eviction leaves holes within and across pages, requiring retained KV entries to be packed into contiguous logical slots. Existing gather operators require additional workspace, while naive in-place compaction introduces read-write conflicts that hinder safe parallelization.

Our key idea is to perform conflict-free slot planning before data movement. We observe that retained entries are typically denser near the tail of allocated pages after eviction. We therefore compact all retained entries into the last X logical blocks. For a request occupying Y logical blocks before compression, the target region is defined by:

$$\begin{aligned} X &= \lceil \text{budget} / \text{block_size} \rceil, \\ \text{start} &= (Y - X) \cdot \text{block_size}, \\ \text{end} &= (Y - X) \cdot \text{block_size} + \text{budget}. \end{aligned}$$

Based on the keep mask, destination slots are evicted positions inside this tail region, while source slots are retained entries outside it:

$$\begin{aligned} \text{dst_slots} &= \text{keep_mask}[\text{start} : \text{end}] == \text{False}, \\ \text{keep_mask}[\text{start} : \text{end}] &= \text{False}, \\ \text{src_slots} &= \text{keep_mask} == \text{True}. \end{aligned}$$

These logical slots are then translated into physical KV cache slots through the slot map.

Algorithm 4 presents our in-place KV cache compaction procedure, triggered only for requests that have just exited the action region, after action attention collection and eviction. Slot planning proceeds in parallel across these requests, pairing retained entries outside the target tail region with

evicted slots inside it and translating logical slots into physical addresses. A customized copy kernel then performs the planned copies in parallel, with each thread block moving one KV slot. Since source and destination slots are disjoint, all copies complete in one pass without an auxiliary KV buffer. Compaction consolidates retained entries into the tail blocks and returns freed blocks to the serving system for reuse. This memory reclamation is particularly important for agentic inference, where token counts can vary significantly across observation-reasoning-action rounds, leading to fluctuating memory demands. The design efficiently converts token-level eviction into reusable paged memory.

8 Experimental Evaluation

8.1 Experimental Methodology

Models. We evaluate ActKV on four recent LLMs with strong agentic capabilities: Qwen3-30B-A3B-Thinking-2507-BF16, Qwen3-235B-A22B-Thinking-2507-FP8 [33], GPT-OSS-20B-BF16, and GPT-OSS-120B-BF16 [20]. These models combine strong reasoning, instruction following, tool use, and structured output, making them representative testbeds for KV compression under agent workloads. We use the official recommended sampling parameters for all models.

Dataset. We evaluate ActKV on three representative agentic benchmarks: ALFWorld [25], HotpotQA-Web [34], and WebShop [35]. ALFWorld requires sequential actions in a simulated household environment based on textual observations. HotpotQA-Web evaluates multi-hop question answering with external Wikipedia [30] retrieval, requiring the model to search, filter, and integrate information across pages. WebShop evaluates web navigation and shopping decisions, where the model compares products and selects an

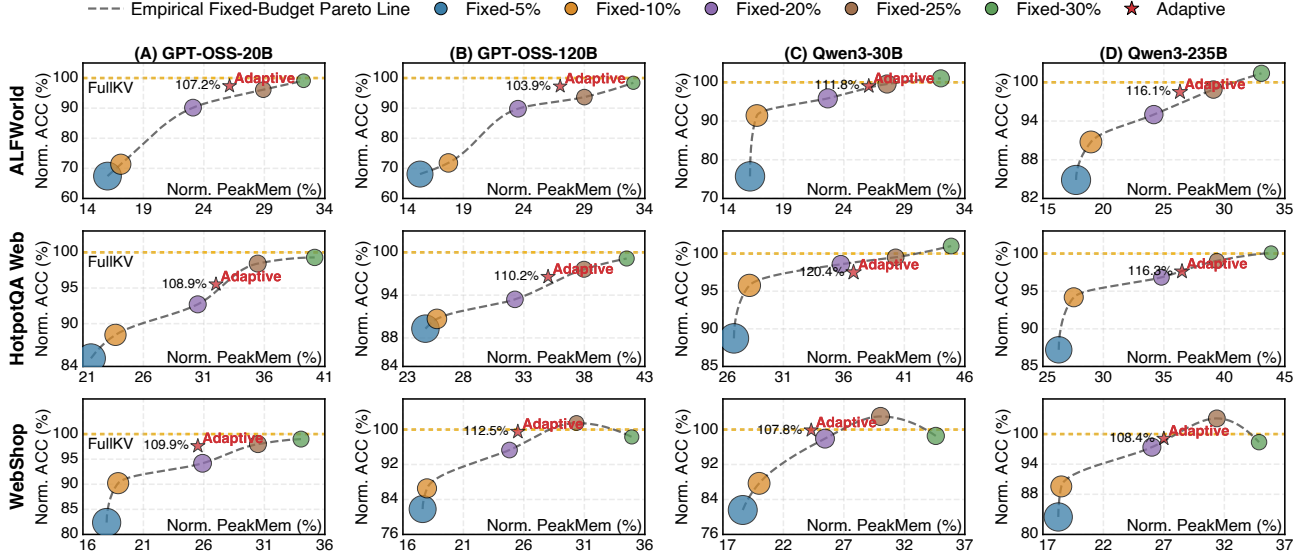


Figure 9. Effectiveness of Adaptive Budget Allocation

item according to the user goal. For each dataset, we sample 1,500 tasks for evaluation.

Agentic Framework. We adopt ReAct [36], a foundational paradigm for modern LLM agents that interleaves observation, reasoning, and action. For ALFWorld, the action space covers environment observation, navigation, receptacle interaction, object manipulation, and object-state transformation, with an average FullKV trace length of 31K tokens. For HotpotQA-Web, actions include searching Wikipedia entries, locating keywords within pages, and submitting final answers, with an average FullKV trace length of 8K tokens. For WebShop, actions support product search, webpage-link clicking, product-page navigation, and purchasing, with an average FullKV trace length of 33K tokens.

Hardware. We conduct experiments on a server equipped with eight NVIDIA RTX PRO 6000 Blackwell GPUs (96 GB), Intel Xeon Platinum 8488C CPUs with 192 cores in total, and 256GB of host memory.

Baselines. We compare ActKV with four representative baselines: FullKV, R-KV [2], SnapKV [17], StreamingLLM [31]. FullKV retains the entire KV cache as an uncompressed reference. SnapKV and StreamingLLM target general long-context generation, while R-KV eliminates redundancy during reasoning. ActKV specifically targets agentic LLM inference. At the implementation level, SnapKV and StreamingLLM primarily provide algorithmic prototypes. R-KV offers a vLLM [27] adaptation, termed paged R-KV, but still gathers KV entries into an auxiliary contiguous workspace outside the paged cache. ActKV supports both Transformers (v5.3.0) and vLLM (v0.19.0), with the latter integrating KV compression directly into paged memory management.

Configuration. For action-oriented eviction, we set the keep-score decay factor $\lambda = 0.5$ and the top- p hit threshold $p = 90\%$. For confidence monitoring, we compute token-level confidence from the top- k predicted tokens with $k = 20$, then apply sliding-window minimum pooling with window size $w = 64$ and stride $s = 8$, followed by bottom- $b\%$ selection with $b = 10$. The adaptive budget starts at 512 entries and increases by a factor of $\text{scale}_{up} = 1.75$ when the fitted confidence slope is negative, capped at $B_{upper} = 30K$ entries. We always preserve the KV entries corresponding to the system prompt and user task description, excluding them from compression to retain essential task instructions.

Metrics. We evaluate five metrics. (1) Accuracy is the fraction of correctly solved tasks, averaged over eight runs per task to mitigate stochastic variability. (2) Budget records the number of KV entries retained after each compression. It remains fixed under static allocation and varies under dynamic allocation. (3) ORA-Len is the number of tokens processed per ORA round. (4) Peak-Memory measures peak KV cache size in entries, given by $\max(\text{Budget} + \text{ORA-Len})$ across rounds for ActKV, since compression occurs only at ORA boundaries, and $\sum \text{ORA-Len}$ for FullKV. (5) Trace-Len is the total number of tokens processed across all rounds of a task, i.e., $\sum \text{ORA-Len}$. We report all metrics relative to their corresponding FullKV values.

8.2 Accuracy Comparison

Fig. 8 compares the accuracy of ActKV and all baselines, normalized to FullKV. We allocate budgets per trace rather than using dataset-wide average trace lengths, accounting for the substantial variation in agentic trace lengths. Although this offline setting requires prior knowledge of trace length,

Table 1. Effectiveness of Paged Compression Kernels

	$r = 1$	$r = 4$	$r = 16$	$r = 64$
1K [0.5K]	3.18×	5.43×	25.05×	25.50×
2K [1.5K]	3.76×	5.16×	15.25×	16.47×
6K [4.0K]	5.63×	5.07×	7.22×	7.18×

it enables a controlled comparison of eviction policies. For each trace and budget ratio r , we first run FullKV to obtain its length T , then run ActKV with budget $B = rT$ and record its mean ORA length $\bar{L}$. ActKV compresses only at ORA boundaries, while each baseline compresses back to the same budget B when its cache exceeds $B + \bar{L}$.

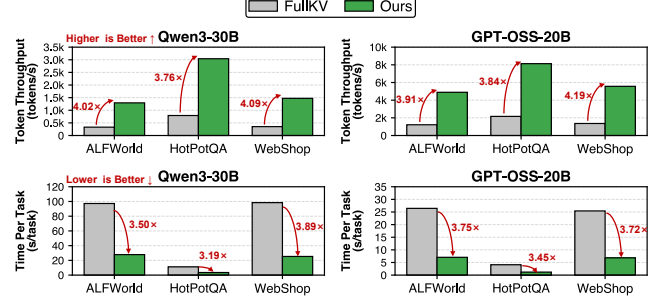
Across all evaluated configurations, ActKV consistently outperforms R-KV, SnapKV, and StreamingLLM. On average, the gains over R-KV, SnapKV, and StreamingLLM are 27.64%, 27.06%, and 54.59% on ALFWorld, 32.86%, 31.88%, and 71.08% on WebShop, and 29.82%, 28.65%, and 70.14% on HotpotQA-Web. On the long-horizon tasks ALFWorld and WebShop, ActKV achieves 81.34% and 101.41% of FullKV accuracy with budgets of only 10% and 25%, suggesting that action-critical information is concentrated in a small subset of KV entries that ActKV effectively preserves.

Preserving action-critical KV entries can turn failed trajectories into successful ones. ActKV improves accuracy over FullKV by up to 1.40% on ALFWorld and 3.13% on WebShop, indicating that selective retention can improve task success while reducing memory usage. As traces grow, FullKV’s expanding cache includes entries less relevant to subsequent actions, potentially diluting attention to action-critical context. These results suggest that ActKV can mitigate this effect by focusing attention on action-critical information, supporting more reliable task progress.

ActKV also generalizes across model families, model scales, and datasets. Its gains are smaller on GPT-OSS than on Qwen3. This difference may partly reflect GPT-OSS’s use of sliding-window attention in half its layers, which could increase sensitivity to further KV eviction. ActKV also maintains high accuracy on Qwen3-235B, where each KV head serves more query heads than in Qwen3-30B, supporting its effectiveness at larger model scales.

8.3 Adaptive Budget Allocation

Fig. 9 compares confidence-driven adaptive and fixed budget allocation in ActKV in terms of accuracy (y-axis), trace length, and peak memory (x-axis), all normalized to FullKV. Per-trace fixed budgets require knowledge of trace lengths from prior FullKV runs and are thus impractical online, whereas adaptive allocation adjusts budgets during inference using confidence feedback. Adaptive ActKV achieves 98.04%, 96.81%, and 99.02% of FullKV accuracy with only 26.38%, 35.08%, and 25.58% of its peak memory on ALFWorld,

**Figure 10.** Token and Task Throughput Comparison

HotpotQA-Web, and WebShop, respectively. Across the long-horizon tasks ALFWorld and WebShop, it averages 98.53% of FullKV accuracy with 25.98% of its peak memory. On both datasets, it lies on the accuracy-memory Pareto frontier [8] among evaluated configurations, demonstrating an effective balance between task success and memory efficiency.

Bubble size denotes trace length, with adaptive ActKV’s values relative to FullKV annotated on the left. Tight budgets, such as 5%, may discard task-critical information, causing persistently low confidence and prolonged reasoning. Since compression occurs only at ORA boundaries, extra reasoning tokens accumulate within each round and may increase peak memory despite a smaller retained budget. ActKV monitors confidence trends and expands the budget upon degradation to mitigate excessive reasoning from insufficient context. Its trace lengths are 109.76%, 113.97%, and 109.67% of FullKV on ALFWorld, HotpotQA-Web, and WebShop, respectively, averaging 109.72% across ALFWorld and WebShop. Adaptive allocation thus achieves substantial memory savings and near-FullKV accuracy with only modest trace growth.

8.4 Paged Compression Kernels

We compare ActKV with paged R-KV to evaluate our customized recovery-based attention calculation and in-place compaction kernels for single compression operations. Regarding memory usage, paged R-KV gathers scattered KV entries into a contiguous workspace, incurring storage and data-copy overhead. ActKV instead operates directly on paged KV layouts without an auxiliary KV buffer. Table 1 reports ActKV’s per-operation speedup over paged R-KV. The experiments use `block_size = 16`, `kv_heads = 8`, and `head_dim = 128`, with r denoting the number of concurrent requests, evaluated at 1, 4, 16, and 64. The notation 6K[4.0K] indicates compressing each request’s KV cache from 6K to 4K entries. Across these settings, ActKV achieves an average 10.41× speedup without additional KV storage.

8.5 End-to-End Throughput

Fig. 10 compares the offline token and task throughput of ActKV and FullKV using Qwen3-30B and GPT-OSS-20B on

a single RTX PRO 6000 Blackwell GPU (96 GB). We submit all 1,500 tasks at once and use the same default vLLM scheduling policy for both methods. ActKV reduces the KV cache footprint through compression and promptly returns freed blocks to the scheduler, allowing memory to be reallocated as requests progress through agentic inference. These mechanisms alleviate memory pressure and enable greater concurrency. Compared with FullKV, ActKV improves token throughput by 3.96 $\times$, 3.80 $\times$, and 4.14 $\times$ on ALFWorld, HotPotQA-Web, and WebShop, respectively. Despite a slight increase in trace length, ActKV also improves task throughput by 3.62 $\times$, 3.32 $\times$, and 3.80 $\times$ on these datasets, respectively.

9 Conclusion

This paper presents ActKV, an action-oriented KV cache management framework for agentic LLM inference. ActKV re-frames cache management around an agent-specific criterion that values each KV entry by its contribution to action generation, shifting the optimization target toward decisions that drive task progress. Guided by this, ActKV combines action-oriented eviction that exploits stable action access patterns, confidence-driven budget allocation that adapts to evolving memory demands, and page-aware kernels that translate token-level eviction into reusable paged memory. Evaluations across diverse models and agentic benchmarks show ActKV preserves near-FullKV accuracy under aggressive compression while substantially improving serving throughput. More broadly, these findings suggest that efficient agent serving should align memory management with task outcomes rather than token-level fidelity, providing a principled foundation for memory-efficient agentic systems.

References

- [1] Anthropic. 2026. Claude Code Overview. <https://code.claude.com/docs/en/overview>. Accessed: 2026-06-09.
- [2] Zefan Cai, Wen Xiao, Hanshi Sun, Cheng Luo, Yikai Zhang, Ke Wan, Yucheng Li, Yeyang Zhou, Li-Wen Chang, Jiuxiang Gu, Zhen Dong, Anima Anandkumar, Abedelkadir Asi, and Junjie Hu. 2025. R-KV: Redundancy-aware KV Cache Compression for Training-Free Reasoning Models Acceleration. In *Advances in Neural Information Processing Systems*.
- [3] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, et al. 2024. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069* (2024).
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Shchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [5] Weijian Chen, Shuibing He, Haoyang Qu, Ruidong Zhang, Siling Yang, Ping Chen, Yi Zheng, Baoxing Huai, and Gang Chen. 2025. IMPRESS: An Importance-Informed Multi-Tier Prefix KV Storage System for Large Language Model Inference. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*.
- [6] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshray Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413* (2025).
- [7] Dao-AILab. 2026. FlashAttention: Fast and Memory-Efficient Exact Attention. <https://github.com/Dao-AILab/flash-attention>. Accessed: 2026-06-10.
- [8] Kalyanmoy Deb. 2001. *Multi-Objective Optimization Using Evolutionary Algorithms*. John Wiley & Sons.
- [9] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2024. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. *arXiv:2411.15100 [cs.CL]* <https://arxiv.org/abs/2411.15100>
- [10] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. 2025. Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. In *Advances in Neural Information Processing Systems (NeurIPS 25)*.
- [11] Yichao Fu, Xuewei Wang, Yuandong Tian, and Jiawei Zhao. 2025. Deep think with confidence. *arXiv preprint arXiv:2508.15260* (2025).
- [12] Junhao Hu, Wenrui Huang, Weidong Wang, Zhenwen Li, Tiancheng Hu, Zhixia Liu, Xusheng Chen, Tao Xie, and Yizhou Shan. 2025. RaaS: Reasoning-Aware Attention Sparsity for Efficient LLM Reasoning. In *Findings of the Association for Computational Linguistics (ACL 2025)*.
- [13] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory OS of AI Agent. *arXiv preprint arXiv:2506.06326* (2025).
- [14] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. 2026. ACON: Optimizing Context Compression for Long-horizon LLM Agents. In *International Conference on Machine Learning*.
- [15] Dominik Kundel. 2025. OpenAI Harmony Response Format. OpenAI Cookbook. Accessed: 2026-09-21. <https://developers.openai.com/cookbook/articles/openai-harmony>
- [16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [17] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. In *Advances in Neural Information Processing Systems*.
- [18] Miao Lu, Weiwei Sun, Weihua Du, Zhan Ling, Xuesong Yao, Kang Liu, and Jiecao Chen. 2025. Scaling LLM Multi-turn RL with End-to-end Summarization-based Context Management. *arXiv preprint arXiv:2510.06727* (2025).
- [19] Elias Lumer, Faheem Nizar, Akshaya Jangiti, Kevin Frank, Anmol Gulati, Mandar Phadate, and Vamse Kumar Subbiah. 2026. Don't Break the Cache: An Evaluation of Prompt Caching for Long-Horizon Agentic Tasks. *arXiv preprint arXiv:2601.06007* (2026).
- [20] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. *arXiv preprint arXiv:2508.10925* (2025).
- [21] OpenClaw. 2026. OpenClaw: Personal AI Assistant. <https://openclaw.ai/>. Accessed: 2026-06-09.
- [22] Qwen Team. [n. d.]. Function Calling. Qwen Documentation. Accessed: 2026-09-21. <https://qwen.readthedocs.io/en/latest/framework/>

- function_call.html
- [23] Akshat Ramachandran, Marina Neseem, Charbel Sakr, Rangharajan Venkatesan, Bruce Khailany, and Tushar Krishna. 2025. ThinKV: Thought-Adaptive KV Cache Compression for Efficient Reasoning Models. *arXiv preprint arXiv:2510.01290* (2025).
 - [24] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv:2302.04761 [cs.CL]* <https://arxiv.org/abs/2302.04761>
 - [25] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *International Conference on Learning Representations*.
 - [26] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. Association for Computing Machinery, 10–19. doi:10.1145/3315508.3329973
 - [27] vLLM Team. 2023. vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention. <https://github.com/vllm-project/vllm>. Accessed: 2026-06-09.
 - [28] Zihan Wang, Cheng Tang, Lei Gong, Cheng Li, Chao Wang, Wenqi Lou, Xuehai Zhou, et al. 2026. Crystal-KV: Efficient KV Cache Management for Chain-of-Thought LLMs via Answer-First Principle. *arXiv preprint arXiv:2601.16986* (2026).
 - [29] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 35. 24824–24837.
 - [30] Wikimedia Foundation. 2026. Wikipedia. <https://www.wikipedia.org/>. Accessed: 2026-06-10.
 - [31] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations*.
 - [32] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. In *Advances in Neural Information Processing Systems*.
 - [33] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
 - [34] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 2369–2380. doi:10.18653/v1/D18-1259
 - [35] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems*, Vol. 35. 20744–20757.
 - [36] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.
 - [37] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems (NeurIPS 23)* 36 (2023), 34661–34710.
 - [38] Youpeng Zhao, Di Wu, and Jun Wang. 2024. Alisa: Accelerating large language model inference via sparsity-aware kv caching. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA 24)*. IEEE, 1005–1017.
 - [39] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2023. SGLang: Efficient Execution of Structured Language Model Programs. *arXiv preprint arXiv:2312.07104* (2023).